Building a Kubernetes Operator with Kubebuilder

Oct 16, 2026 · @Ming

This guide builds a small operator that defines a `Website` custom resource. You declare an image and a replica count, and the controller creates and maintains a Deployment and a NodePort Service for it, continuously correcting any drift.

1. Scaffold the project

You need Go, `kubebuilder`, `kubectl`, and a cluster such as kind or minikube.

```
mkdir website-operator && cd website-operator
kubebuilder init --domain example.com --repo example.com/website-operator
kubebuilder create api --group web --version v1 --kind Website
# answer "y" to both "Create Resource" and "Create Controller"
```

This produces the two files you edit: `api/v1/website_types.go` (the CRD's schema) and `internal/controller/website_controller.go` (the reconcile logic).

2. Define the CRD

Edit `api/v1/website_types.go`. The `// +kubebuilder:` comments are **markers**: Kubebuilder reads them to generate validation rules, defaults, and `kubectl` columns into the CRD YAML.

```
package v1

import (
        metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

// WebsiteSpec is the desired state.
type WebsiteSpec struct {
        // +kubebuilder:default="nginx:1.27"
        Image string `json:"image,omitempty"`

        // +kubebuilder:validation:Minimum=1
```

```
        // +kubebuilder:validation:Maximum=10
        // +kubebuilder:default=1
        Replicas int32 `json:"replicas,omitempty"`

        // +kubebuilder:validation:Minimum=30000
        // +kubebuilder:validation:Maximum=32767
        // +optional
        NodePort int32 `json:"nodePort,omitempty"`
}

// WebsiteStatus is the observed state, written by the controller.
type WebsiteStatus struct {
        ReadyReplicas int32              `json:"readyReplicas,omitempty"`
        Conditions    []metav1.Condition `json:"conditions,omitempty"`
}

// +kubebuilder:object:root=true
// +kubebuilder:subresource:status
// +kubebuilder:printcolumn:name="Image",type=string,JSONPath=`.spec.image`
//
+kubebuilder:printcolumn:name="Replicas",type=integer,JSONPath=`.spec.replica
s`
//
+kubebuilder:printcolumn:name="Ready",type=integer,JSONPath=`.status.readyRep
licas`
type Website struct {
        metav1.TypeMeta   `json:",inline"`
        metav1.ObjectMeta `json:"metadata,omitempty"`

        Spec   WebsiteSpec   `json:"spec,omitempty"`
        Status WebsiteStatus `json:"status,omitempty"`
}

// +kubebuilder:object:root=true
type WebsiteList struct {
        metav1.TypeMeta `json:",inline"`
        metav1.ListMeta `json:"metadata,omitempty"`
        Items           []Website `json:"items"`
}

func init() {
        SchemeBuilder.Register(&Website{}, &WebsiteList{})
}
```

Spec and status are kept separate. Users write `spec`; only the controller writes `status`. The `subresource:status` marker enforces that split at the API level.

3. Write the controller

Replace the scaffolded `Reconcile` in `internal/controller/website_controller.go`:

```
package controller

import (
        "context"

        appsv1 "k8s.io/api/apps/v1"
        corev1 "k8s.io/api/core/v1"
        apierrors "k8s.io/apimachinery/pkg/api/errors"
        "k8s.io/apimachinery/pkg/api/meta"
        metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
        "k8s.io/apimachinery/pkg/runtime"
        "k8s.io/apimachinery/pkg/util/intstr"
        ctrl "sigs.k8s.io/controller-runtime"
        "sigs.k8s.io/controller-runtime/pkg/client"
        "sigs.k8s.io/controller-runtime/pkg/controller/controllerutil"
        "sigs.k8s.io/controller-runtime/pkg/log"

        webv1 "example.com/website-operator/api/v1"
)

type WebsiteReconciler struct {
        client.Client
        Scheme *runtime.Scheme
}

//
+kubebuilder:rbac:groups=web.example.com,resources=websites,verbs=get;list;wa
tch;update;patch
//
+kubebuilder:rbac:groups=web.example.com,resources=websites/status,verbs=get;
update;patch
//
+kubebuilder:rbac:groups=apps,resources=deployments,verbs=get;list;watch;crea
te;update;patch;delete
//
```

```
+kubebuilder:rbac:groups="",resources=services,verbs=get;list;watch;create;up
date;patch;delete

func (r *WebsiteReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
        logger := log.FromContext(ctx)

        // 1. Fetch the Website. If it was deleted, there's nothing to do:
        //    the Deployment and Service are garbage-collected via owner
references.
        var site webv1.Website
        if err := r.Get(ctx, req.NamespacedName, &site); err != nil {
                return ctrl.Result{}, client.IgnoreNotFound(err)
        }

        labels := map[string]string{"app": site.Name}

        // 2. Make the Deployment match the spec (create it, or update it if
it drifted).
        deploy := &appsv1.Deployment{
                ObjectMeta: metav1.ObjectMeta{Name: site.Name, Namespace:
site.Namespace},
        }
        op, err := controllerutil.CreateOrUpdate(ctx, r.Client, deploy,
func() error {
                replicas := site.Spec.Replicas
                deploy.Spec.Replicas = &replicas
                deploy.Spec.Selector = &metav1.LabelSelector{MatchLabels:
labels}
                deploy.Spec.Template.ObjectMeta.Labels = labels
                deploy.Spec.Template.Spec.Containers = []corev1.Container{{
                        Name:  "web",
                        Image: site.Spec.Image,
                        Ports: []corev1.ContainerPort{{ContainerPort: 80}},
                }}
                // Owner reference: deleting the Website deletes this
Deployment.
                return controllerutil.SetControllerReference(&site, deploy,
r.Scheme)
        })
        if err != nil {
                return ctrl.Result{}, err // returning an error requeues with
backoff
```

```
        }
        logger.Info("deployment reconciled", "operation", op)

        // 3. Same for the Service.
        svc := &corev1.Service{
                ObjectMeta: metav1.ObjectMeta{Name: site.Name, Namespace:
site.Namespace},
        }
        _, err = controllerutil.CreateOrUpdate(ctx, r.Client, svc, func()
error {
                svc.Spec.Type = corev1.ServiceTypeNodePort
                svc.Spec.Selector = labels
                port := corev1.ServicePort{Port: 80, TargetPort:
intstr.FromInt32(80)}
                if len(svc.Spec.Ports) > 0 {
                        port.NodePort = svc.Spec.Ports[0].NodePort // keep
the one Kubernetes assigned
                }
                if site.Spec.NodePort != 0 {
                        port.NodePort = site.Spec.NodePort
                }
                svc.Spec.Ports = []corev1.ServicePort{port}
                return controllerutil.SetControllerReference(&site, svc,
r.Scheme)
        })
        if err != nil {
                return ctrl.Result{}, err
        }

        // 4. Report observed state back in status.
        site.Status.ReadyReplicas = deploy.Status.ReadyReplicas
        ready := metav1.ConditionFalse
        reason := "Progressing"
        if deploy.Status.ReadyReplicas == site.Spec.Replicas {
                ready, reason = metav1.ConditionTrue, "AllReplicasReady"
        }
        meta.SetStatusCondition(&site.Status.Conditions, metav1.Condition{
                Type:               "Ready",
                Status:             ready,
                Reason:             reason,
                ObservedGeneration: site.Generation,
        })
        if err := r.Status().Update(ctx, &site); err != nil {
```

```
                if apierrors.IsConflict(err) {
                        return ctrl.Result{Requeue: true}, nil // someone
else updated it; try again
                }
                return ctrl.Result{}, err
        }

        return ctrl.Result{}, nil
}

func (r *WebsiteReconciler) SetupWithManager(mgr ctrl.Manager) error {
        return ctrl.NewControllerManagedBy(mgr).
                For(&webv1.Website{}).      // reconcile when a Website
changes
                Owns(&appsv1.Deployment{}). // ...or when a Deployment it
owns changes
                Owns(&corev1.Service{}).    // ...or a Service it owns
                Named("website").
                Complete(r)
}
```

Design points

- **Level-triggered, not event-driven.** `Reconcile` never asks what changed. It reads current state and makes it match the spec, every time. That is why one function handles creation, updates, a manually deleted Deployment, and a controller restart.
- **Idempotent.** `CreateOrUpdate` makes running `Reconcile` ten times in a row have the same effect as running it once. That matters because it does get called repeatedly.
- **`Owns()` closes the loop.** If someone runs `kubectl scale deployment my-site --replicas=5`, the Deployment change triggers a reconcile of its owner, and the controller sets it back to the `Website`'s replica count.
- **RBAC markers** generate the ClusterRole the controller needs. A missing one shows up as `forbidden` errors in the logs.

4. Run it

```
make manifests   # regenerate CRD YAML and RBAC from the markers
make install     # install the CRD into the cluster
make run         # run the controller locally against your kubeconfig
```

In another terminal, create a `Website`:

```
# website.yaml
apiVersion: web.example.com/v1
kind: Website
metadata:
  name: my-site
spec:
  image: nginx:1.27
  replicas: 2
  nodePort: 30201
```

```
kubectl apply -f website.yaml
kubectl get websites            # shows Image / Replicas / Ready columns
kubectl get deploy,svc my-site  # created by your controller
```

Self-healing demo

```
kubectl scale deploy my-site --replicas=5   # controller sets it back to 2
kubectl delete deploy my-site               # controller recreates it
kubectl delete website my-site              # Deployment and Service are
garbage-collected
```

Validation works without any code: `replicas: 50` is rejected by the API server because of the `Maximum=10` marker.

5. Where the concurrency shows up

Shutdown is a context, which is a closed channel

In `cmd/main.go`, the scaffold calls `mgr.Start(ctrl.SetupSignalHandler())`. On SIGTERM, that context's `Done()` channel is closed — the same close-to-broadcast pattern as an `ExitChan`. Every controller, informer, and runnable sees it and stops.

Your own background goroutines are Runnables

```
type heartbeat struct {
        interval time.Duration
}
```

```
func (h *heartbeat) Start(ctx context.Context) error {
        logger := log.FromContext(ctx)
        ticker := time.NewTicker(h.interval)
        defer ticker.Stop()

        for {
                select {
                case <-ctx.Done(): // manager is shutting down
                        logger.Info("heartbeat stopping")
                        return nil
                case <-ticker.C:
                        logger.Info("operator alive")
                }
        }
}

// in cmd/main.go, before mgr.Start:
if err := mgr.Add(&heartbeat{interval: 30 * time.Second}); err != nil {
        setupLog.Error(err, "unable to add heartbeat")
        os.Exit(1)
}
```

Worker concurrency is configurable

By default one goroutine processes reconciles. To run several in parallel:

```
import "sigs.k8s.io/controller-runtime/pkg/controller"

return ctrl.NewControllerManagedBy(mgr).
        For(&webv1.Website{}).
        Owns(&appsv1.Deployment{}).
        WithOptions(controller.Options{MaxConcurrentReconciles: 3}).
        Complete(r)
```

The **work queue** behind this is a producer/consumer setup with two extra guarantees: the same object is never reconciled by two workers at once, and repeated events for one object are deduplicated into a single reconcile. So `Reconcile` needs no per-object locking — it just needs to be idempotent.

Retry with backoff is built in

Returning an error puts the request back on the queue with exponential backoff. `ctrl.Result{RequeueAfter: time.Minute}` schedules a periodic recheck, useful when polling something outside the cluster.

6. Ideas for extending it

1. **Add a finalizer** so the controller runs cleanup logic (for example, deleting something outside the cluster) before Kubernetes removes the `Website`.
2. **Write tests with envtest.** The scaffold includes `internal/controller/suite_test.go`, which starts a real API server and etcd without a full cluster. Test that creating a `Website` produces a Deployment with the right replica count.
3. **Add a validating webhook** (`kubebuilder create webhook`) for rules the schema markers can't express, such as requiring images from your own registry.
4. **Deploy it into the cluster** with `make docker-build docker-push IMG=...` and `make deploy IMG=...` instead of `make run`, and enable leader election so only one replica is active at a time.
5. **Add a `v2` API version with a conversion webhook** to demonstrate CRD versioning.

Put the project on GitHub with a README showing the self-healing demo from section 4. Being able to explain why `Reconcile` is level-triggered and idempotent is the depth interviewers look for.